CatMow 开发日志

CatMow 开发日志：从零到 Pi5 部署

> 记录两天内训练猫声识别模型、部署到 Raspberry Pi 5 的完整过程，包括遇到的问题和解决方案。

一、项目背景与目标

为猫咪声音实时识别系统（运行在 Raspberry Pi 5 上）训练一个深度学习分类器，能够区分猫的不同发声状态，以便做出相应的智能响应（如开门、补食、安抚等）。

最终系统架构：

```
麦克风 → VAD 预过滤 → Mel频谱图 → MobileNetV2 → 4类分类输出
                                                  meow / purr / yowl / other
```

二、模型架构演进：7类 → 3类 → 4类

版本	类别	问题
初版	7类（细粒度）	数据严重不均衡，类间边界模糊
v1	3类（meow/purr/hiss）	hiss 与背景噪音高度重叠，大量误报
最终	**4类（meow/purr/yowl/other）**	✅ 合理的类别划分

类别说明： – `meow` — 猫叫（含 hungry/happy 子集） – `purr` — 呼噜声（低频连续） – `yowl` — 嚎叫/激动发声（含精选 growl） – `other` — 背景噪音/静音

模型架构： – Base: MobileNetV2（ImageNet 预训练） – 输入: 1s 音频 → 128×128×3 Mel 频谱图 – 两阶段训练：Phase 1 冻结基础层训练分类头；Phase 2 解冻顶部 30 层微调 – 输出: TFLite（~9.8 MB），适合 Pi5 边缘推理

三、数据准备

数据来源与映射

```
SOURCE_TO_LABEL = {
    "meow":       0,   # meow
    "hungry":     0,   # → meow
    "happy":      0,   # → meow
    "purr":       1,   # purr
```

```
    "growl":      2,   # → yowl (精选高质量文件)
    "yowl":       2,   # yowl (11段90s录音)
    "background": 3,   # other
    # hiss 已移除 — 频谱与 purr 重叠, 污染 yowl 类
}
```

v2 训练集规模

类别	训练样本	验证样本	测试样本
meow	10,698	408	482
purr	7,926	398	74
yowl	7,956	363	384
other	2,142	119	119
合计	**28,722**	**1,288**	**1,059**

四、遇到的主要问题与解决过程

问题 1：程序启动崩溃（KeyError: ‘hiss’）

现象： 部署 v1 模型到 Pi5 后，`pi5_inference.py` 启动时直接崩溃退出。

根因： 代码中打印各类阈值的语句还引用了已被删除的 `CLASS_THRESHOLDS['hiss']`。

修复： 一行代码修复，将 `'hiss'` 改为 `'yowl'` 和 `'other'`。

问题 2：Purr 以 93–100% 置信度被误判为 Yowl

现象： v1 模型实测，播放 purr 音频时，前几分钟几乎全部输出 YOWL，置信度高达 93–100%。

排查： 查看 yowl 训练数据构成：

```
yowl 类 = yowl(11个录音) + growl(31个文件) + hiss(31个文件)
```

根因： hiss（嘶嘶声）是宽频低沉声，**频谱与 purr 高度重叠**。模型学到”宽频低频 → yowl”，而 purr 恰好也是宽频低频，于是全被判为 yowl。

解决方案：三步清洗 yowl 训练数据

① **完全移除 hiss**：从 `SOURCE_TO_LABEL` 删除 `"hiss": 2` 映射

② **Growl 质量过滤**（`scripts/filter_growl.py`）： – 对31个 growl 文件按频谱强度评分 – 评分公式：`centroid_mean × (1 + centroid_std/centroid_mean) / (flatness × 10000 + 1)` – 保留前15名（高频、强调制、低平坦度 → 更像嚎叫） – 归档后16名（高平坦度 → 宽频噪音，与 purr 重叠）

③ **重训练**：v2 模型 val_accuracy 从 71% 提升到 **86.5%**

问题 3：Purr 完全无输出

现象： VAD 和阈值调整后，播放 purr 音频仍然几乎没有任何输出。

排查： 检查 VAD（语音活动检测）预处理逻辑，发现**在声音进入模型之前就被过滤掉了**。

根因： VAD 中"猫声频带"设为 300–3500 Hz，但 purr 的基频在 **25–150 Hz**，大部分能量落在"噪音频带"(< 300 Hz) 里，导致 `cat_energy / noise_energy < 2.0`，直接跳过不送入模型。

```
# 修改前 ❌
cat_band = (freqs >= 300) & (freqs <= 3500)
noise_band = (freqs < 300) | (freqs > 3500)
if cat_energy / noise_energy < 2.0: continue  # purr 在这里被过滤!

# 修改后 ✅
cat_band = (freqs >= 100) & (freqs <= 3500)   # 下限 300Hz → 100Hz
noise_band = (freqs < 100) | (freqs > 3500)
if cat_energy / noise_energy < 1.5: continue   # 阈值 2.0 → 1.5
```

效果： 用户验证"调整了 VAD 之后，purr 确实出结果了" ✅

教训： 频率特性必须与算法设计一致。Purr 基频 25–150 Hz 这个**物理事实**，直接决定了 VAD 频带的设计边界。不了解这个，调再多参数也解决不了根本问题。

问题 4：模型过于"自信"导致误报

现象： 几乎所有预测置信度都在 85–100%，即使分类错误也如此（过度自信）。

解决：调整各类别阈值

```
CLASS_THRESHOLDS = {
    "meow": 0.60,   # 不变
    "purr": 0.55,   # 降低 (0.65 → 0.55), 提高召回率
    "yowl": 0.88,   # 提高 (0.75 → 0.88), 减少误报
    "other": 0.50,  # 不变
}
```

问题 5：Purr 仍然被误判为 Meow（未完全解决）

现象： VAD 修复后 purr 有输出了，但相当一部分被识别为 meow。

分析： – Purr 训练数据全部来自 YouTube 视频（8段），音频特征趋于一致，**diversity 严重不足** – 原始 mp3 文件和新录的 90s 片段音频特征差异大，训练/测试分布不匹配 – 1s 窗口有时切到 purr 的过渡段，特征模糊

现状： 问题已定位，根本解决需要采集更多来源多样的 purr 数据（不同猫、不同环境、真实猫咪）。这是下一步的工作。

五、工程与部署问题

工程问题 1：训练耗时长，需要后台运行与进度监控

背景： 训练一次约需 30–60 分钟，不能阻塞终端。

解决方案： 使用后台进程 + 日志文件监控：

```
# 后台运行训练，输出重定向到日志
/opt/homebrew/bin/python3.11 scripts/03_train.py > /tmp/train_v2.log 2>&1 &

# 实时查看训练进度（不打断训练）
tail -f /tmp/train_v2.log

# 确认进程还在运行
ps aux | grep train
```

教训： 长时间任务必须后台化 + 持久化日志，随时可以脱离终端观察进度，也方便事后追溯最佳 epoch。

工程问题 2：录音脚本 `record_yowl.py` 三个 Bug 导致无法录音

背景： 为了采集 yowl 数据，在 Pi5 上编写了录音脚本，但运行后程序卡死、Ctrl+C 无法退出、文件从未保存。

三个 Bug：

① **主线程卡死**：使用了 `sd.InputStream` + callback + `threading.Event` 的方式。sounddevice 的 PortAudio 回调线程与 Python 主线程之间存在 GIL 冲突，`done_event.set()` 触发后主线程的 while 循环无法退出。

② **q 键退出无效**：程序卡在 while 循环时，永远无法到达 `input("q退出")` 这一行。

③ **Ctrl+C 强制退出时文件未保存**：`wavfile.write()` 在 while 循环之后，被强制退出直接跳过了。

修复方案：用 `sd.rec()` 替代 callback 方式

```
# 修复后的核心逻辑
recording = sd.rec(total_frames, samplerate=SAMPLE_RATE, channels=1)
try:
    while not done:
        elapsed = time.time() - start_time
        rms = np.sqrt(np.mean(recording[:frames_recorded] ** 2))
        print(f"\r[{elapsed:.1f}s] RMS: {rms:.4f}", end="")
        time.sleep(0.5)
    sd.wait()
except KeyboardInterrupt:
    sd.stop()                    # Ctrl+C 时先停录音
    wavfile.write(path, ...)     # 再保存已录内容
    print("提前结束，文件已保存")
```

效果： 成功在 Pi5 上录制了 13 段 yowl 音频（每段 90s），筛选保留 11 段。

工程问题 3：数据在 Pi5 和 Mac 之间的传输

场景： 在 Pi5 上录音 → 传回 Mac 训练 → 训练好的模型传到 Pi5 推理。

使用 scp 双向传输：

```
# Pi5 → Mac：拉取录音数据
scp -r xiaoyan@10.0.0.16:~/catmow/yowl_recordings/ data/raw/yowl/

# Mac → Pi5：部署新模型和推理脚本
scp models/cat_sound.tflite pi5/pi5_inference.py xiaoyan@10.0.0.16:~/catmow/
```

问题： 每次修改推理脚本后都要手动 scp，容易忘记，导致 Pi5 上跑的是旧版本。

教训： 多设备开发场景下，部署步骤应该固化成脚本（`deploy.sh`），避免手动操作遗漏。

工程问题 4：TFLite 转换与验证

背景： Keras 模型需要转换为 TFLite 才能在 Pi5 上高效推理。

转换流程：

```
converter = tf.lite.TFLiteConverter.from_saved_model(saved_model_dir)
converter.optimizations = [tf.lite.Optimize.DEFAULT]   # 量化优化
tflite_model = converter.convert()
# 输出：cat_sound.tflite，9.83 MB
```

验证步骤： 转换后用前100条测试样本对比 Keras 和 TFLite 输出，确认量化误差在可接受范围内：

```
TFLite 验证（前100条）：meow=100%, purr=70%, yowl=25%, other=5%
```

教训： 量化会损失精度，转换后必须验证，不能直接假设结果与 Keras 一致。

工程问题 5：大文件管理与 Git 仓库设计

问题： 项目目录总大小约 6GB，但其中绝大部分是中间产物：

目录	大小	说明
`data/features/`	5.7 GB	numpy 预处理缓存（X.npy, y.npy）
`models/`	108 MB	Keras 模型文件
`data/processed/`	227 MB	原始音频切片
`models/cat_sound.tflite`	9.8 MB	最终推理模型
`scripts/ + pi5/`	56 KB	核心代码

策略： `.gitignore` 只排除不必要的大文件，保留真正有价值的内容：

```
data/features/          # 可随时重新生成，不上传
data/raw/               # 音频原文件较大，不上传
models/*.keras          # Keras 格式较大，只上传 .tflite
models/saved_model_export/
```

最终上传内容： scripts/ + pi5/ + data/processed/ + cat_sound.tflite = **~189 MB**，符合 GitHub 限制。

工程问题 6：GitHub 上传三连坑

这是工程经历中最曲折的部分，踩了三个独立的坑。

坑1：GitHub 已停止密码认证（403 Permission Denied）

2021 年 8 月起，GitHub 不再接受账号密码进行 git 操作。输入密码会报：

```
remote: Permission to dxy03rico/CatMow.git denied to dxy03rico.
fatal: unable to access: The requested URL returned error: 403
```

解决： 必须使用 Personal Access Token（PAT）。在 github.com/settings/tokens 生成，勾选 `repo` 权限，将 token 作为"密码"输入。

坑2：macOS 系统 git 的 LibreSSL SSL 错误

使用 PAT 后仍然失败：

```
error: RPC failed; curl 55 LibreSSL SSL_read:
  error:1404C3FC:SSL routines:ST_OK:sslv3 alert bad record mac
fatal: the remote end hung up unexpectedly
```

根因： macOS 自带的 git（`/usr/bin/git`）使用 LibreSSL，在传输大文件（~189MB）时有已知的 SSL 握手 bug。

解决： 安装 Homebrew 版 git（使用 OpenSSL），切换后立即成功：

```
brew install git
/opt/homebrew/bin/git push -u origin main   # ✅ 成功
```

长期解决方案：将 Homebrew git 加入 PATH 优先级：

```
echo 'export PATH="/opt/homebrew/bin:$PATH"' >> ~/.zshrc
```

坑3：`Everything up-to-date` 误导性提示

在 LibreSSL 错误发生后，git 输出了令人困惑的 `Everything up-to-date`，让人以为 push 成功了。

验证方法： 不能只看 git 提示，要用 `git ls-remote origin` 确认远端实际状态：

```
git ls-remote origin
# 空输出 → 远端确实为空，push 失败
# 有 commit hash → push 成功
```

教训： 工程操作要验证结果，不要只信工具的输出信息。

六、v1 vs v2 训练结果对比

指标	v1	v2
Val Accuracy	71%	**86.5%** ↑
Val Loss	0.515	**0.433** ↓
Test Accuracy	76%	75%
Meow Test	95.4%	91.3%
Purr Test	58.1%	46.0%*
Yowl Test	78.5%	82.6% ↑
Other Test	0.0%	2.5%
实际体感（purr→yowl）	严重误判	基本正常 ✅

> *Purr 测试集仅 74 条样本，且来自与训练集不同的分布（原始 mp3），不代表真实场景。Val accuracy 显著提升，实测效果更准确地反映了模型质量。

七、系统技术细节

VAD 三级预过滤

```
Level 1: RMS < 0.01           → 静音，跳过
Level 2: cat/noise ratio < 1.5 → 非猫声频带，跳过
Level 3: spectral flatness > 0.5 → 宽频噪音（空调/风扇），跳过
```

防抖机制

参数	值	含义
SUSTAIN_FRAMES	2（≈1s）	连续N帧才触发输出
COOLDOWN_SAME	3.0s	同类事件冷却
COOLDOWN_DIFF	0.5s	不同类事件冷却

音频预处理参数

参数	值
采样率	22050 Hz
窗口长度	1s

参数	值
跳步	0.5s
Mel bins	128
FFT size	1024
Hop length	256

八、经验总结

1. **先验证声音能到达模型**：VAD 过滤问题比模型本身更”上游”。被 VAD 干掉的声音，再怎么调模型参数也没用。遇到”完全没输出”的情况，先排查预处理，而不是模型。

2. **数据构成比模型结构更重要**：Yowl 类混入 hiss 后，模型学到了错误的频谱特征，这不是超参数问题，只能靠数据清洗解决。垃圾进垃圾出，数据质量永远第一。

3. **物理特性决定算法边界**：Purr 基频 25–150 Hz 这个猫咪生理特性，直接决定了 VAD 频带设计。做音频 AI 需要先理解声学，再做工程。

4. **测试指标 vs 实际体感**：v2 整体 test accuracy 略低于 v1（75% vs 76%），但实测效果明显更好。测试集代表性不足时，要相信实测，不要盲目相信数字。

5. **小数据集要做质量过滤**：与其有多少用多少，不如用 spectral scoring 等方法只保留高质量样本。15个精选 growl 比 31个混杂的更有价值。

6. **边缘部署需要分层设计**：模型本身只是系统的一部分。VAD 预过滤、置信度阈值、防抖机制共同决定了最终体验。每一层都需要根据实际场景调整。

九、下一步方向

- [ ] 采集更多多样化的 purr 录音（真实猫咪、不同场景、不同录音设备）
- [ ] 扩充 other 类数据（当前测试准确率仅 2.5%）
- [ ] 考虑在线学习：Pi5 上通过用户反馈持续微调
- [ ] 探索更轻量的模型（EfficientNet-Lite / 自定义小型 CNN）进一步降低推理延迟

项目仓库：https://github.com/dxy03rico/CatMow